Étude expérimentale d'un ABR

Benchmark de l'insertion, de la recherche et de la suppression

ERRAMI Youssef – Programmation C++

Résumé

Ce travail étudie un arbre de recherche binaire (ABR) implémenté en C++. Le programme mesure le temps total d'insertion, de recherche et de suppression pour neuf tailles d'entrée, comprises entre 1 000 et 1 000 000 valeurs. Toutes les valeurs testées ont été retrouvées. Les résultats sont des mesures réelles d'une exécution par taille, et non des moyennes de plusieurs répétitions.

Configuration utilisée

Ordinateur	MSI Stealth GS66 12UGS
Processeur	Intel Core i7-12700H, 14 cœurs
Carte graphique	NVIDIA GeForce RTX 3070 Ti
Mémoire vive	16 Go DDR5
Langage	C++17 ; les calculs de l'ABR sont exécutés sur le processeur.

Protocole de mesure

Le programme construit les entiers de 0 à $n-1$, les mélange, puis chronomètre les trois opérations avec `std::chrono::steady_cloc`
La recherche porte sur les mêmes valeurs que celles insérées : ce sont donc des recherches réussies. Les durées sont des temps cumulés en millisecondes pour les n opérations. Le parcours écrit dans `output.txt` est effectué hors des intervalles chronométrés.

Le programme a été compilé sans option d'optimisation explicite avec `g++ -std=c++17 -Wall -Wextra -pedantic`. Les neuf tailles ont été testées successivement dans le même lancement ; chaque taille correspond à un nouvel arbre et à un ordre de valeurs mélangé.

Mesures obtenues

Taille n	Recherches réussies	Insertion (ms)	Recherche (ms)	Suppression (ms)
1 000	1 000 / 1 000	0,126	0,064	0,202
3 333	3 333 / 3 333	0,920	0,624	0,677
10 000	10 000 / 10 000	3,253	1,518	1,839
50 000	50 000 / 50 000	12,830	6,824	9,671
100 000	100 000 / 100 000	28,445	20,958	28,501
250 000	250 000 / 250 000	127,151	94,481	115,091
500 000	500 000 / 500 000	308,352	273,892	289,340
777 777	777 777 / 777 777	642,876	576,837	595,468
1 000 000	1 000 000 / 1 000 000	975,218	845,118	909,071

Structure et règle de l'arbre

Chaque élément, appelé *noeud*, contient une valeur et deux pointeurs : `gauche` et `droite`. Pour chaque noeud, toutes les valeurs du sous-arbre gauche sont plus petites que sa valeur et celles du sous-arbre droit sont plus grandes. Cette règle permet de choisir une seule branche à chaque étape d'une recherche.

```
struct Noeud {
    int valeur;
    Noeud* gauche;
    Noeud* droite;

    Noeud(int nouvelleValeur) {
        valeur = nouvelleValeur;
        gauche = nullptr;
        droite = nullptr;
    }
};
```

Insertion

L'insertion commence à la racine. Si le pointeur courant est nul, on crée un nouveau noeud à cet emplacement. Sinon, on compare la valeur à insérer avec celle du noeud courant : une valeur plus petite continue à gauche, une valeur plus grande continue à droite. Dans cette implémentation, une valeur déjà présente n'est pas ajoutée une deuxième fois.

```
Noeud* inserer(Noeud* racine, int valeur) {
    if (racine == nullptr)
        return new Noeud(valeur);

    if (valeur < racine->valeur)
        racine->gauche = inserer(racine->gauche, valeur);
    else if (valeur > racine->valeur)
        racine->droite = inserer(racine->droite, valeur);
    else
        cout << "Cette valeur existe deja dans l'arbre.\\n";

    return racine;
}
```

La fonction est récursive : chaque appel traite un niveau supplémentaire de l'arbre. Le pointeur de racine retourné permet de conserver la racine initiale, ou de recevoir le nouveau noeud lorsqu'on insère dans un arbre vide. Le benchmark génère des valeurs distinctes, donc le cas des doublons n'intervient pas dans ces mesures.

Recherche

La recherche compare la valeur cherchée au noeud courant. Une égalité signifie que la valeur est trouvée. Sinon, la comparaison dirige la recherche vers le sous-arbre gauche ou droit. Si le pointeur devient nul, la valeur n'existe pas dans l'arbre.

```
bool rechercher(Noeud* racine, int valeur) {
    if (racine == nullptr)
        return false;
    if (valeur == racine->valeur)
        return true;
    if (valeur < racine->valeur)
        return rechercher(racine->gauche, valeur);
    return rechercher(racine->droite, valeur);
}
```

La recherche du test est favorable au sens où chaque requête correspond à une valeur présente. Le temps rapporté est le cumul de n appels, dans l'ordre mélangé des valeurs, et non la durée d'une seule recherche. La suppression reprend ensuite cette même liste mélangée.

Suppression

La suppression conserve la règle de classement de l'ABR et distingue trois cas :

1. **Aucun enfant :** le noeud est supprimé et remplacé par nul.
2. **Un seul enfant :** le noeud est supprimé et son enfant prend sa place.
3. **Deux enfants :** la valeur du noeud est remplacée par son successeur, le minimum du sous-arbre droit ; ce successeur est ensuite supprimé à son emplacement d'origine.

```
Noeud* supprimer(Noeud* racine, int valeur) {
    if (racine == nullptr) return nullptr;
    if (valeur < racine->valeur)
        racine->gauche = supprimer(racine->gauche, valeur);
    else if (valeur > racine->valeur)
        racine->droite = supprimer(racine->droite, valeur);
    else if (racine->gauche == nullptr) {
        Noeud* droite = racine->droite;
        delete racine;
        return droite;
    } else if (racine->droite == nullptr) {
        Noeud* gauche = racine->gauche;
        delete racine;
        return gauche;
    } else {
        Noeud* min = trouverMinimum(racine->droite);
        racine->valeur = min->valeur;
        racine->droite = supprimer(racine->droite, min->valeur);
    }
    return racine;
}
```

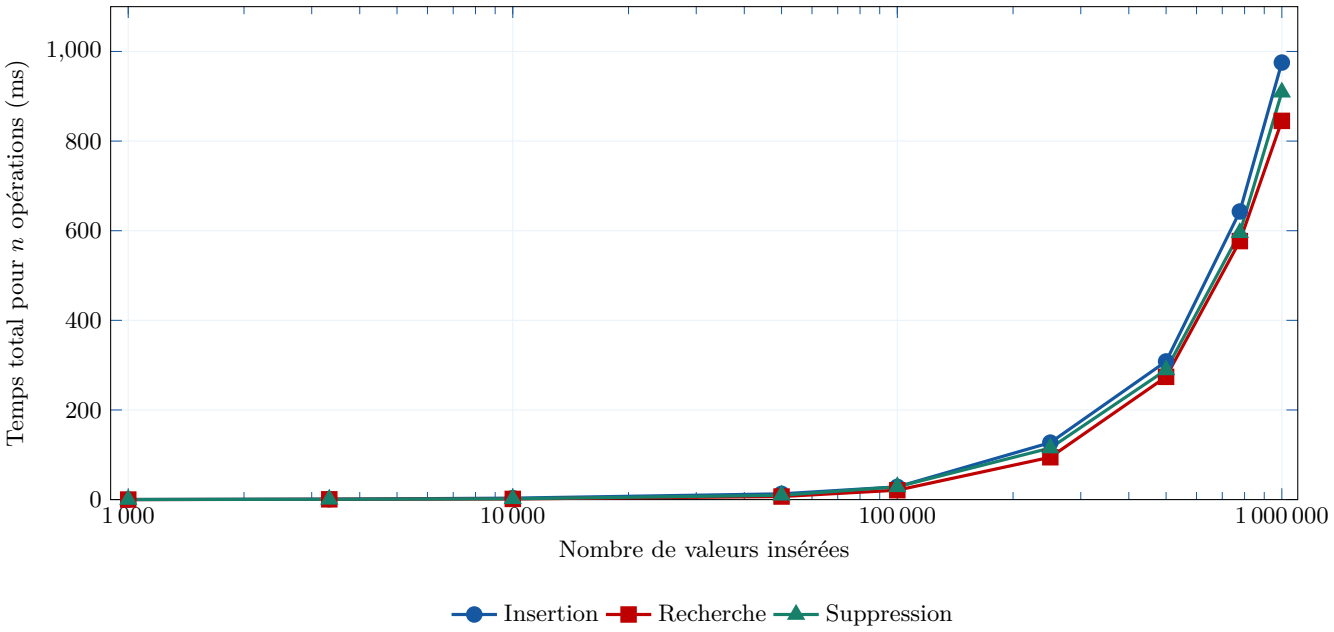
Affichage et gestion de la mémoire

Le parcours *gauche–racine–droite* affiche les valeurs en ordre croissant. Pendant le test, le résultat est dirigé vers `output.txt` pour éviter d'imprimer jusqu'à un million de valeurs dans le terminal. Après le benchmark, les valeurs du test sont supprimées ; la fonction `libererArbre` libère également les noeuds restants avec `delete`.

Chronométrage

Pour chaque opération, le programme lit l'heure juste avant la boucle et juste après. La différence avec `steady_clock` donne une durée monotone, exprimée en millisecondes. Les générations, le mélange et l'écriture du fichier ne sont pas inclus dans les intervalles chronométrés. En revanche, la création et la destruction des noeuds font partie des temps d'insertion et de suppression, comme dans le code testé.

Comparaison graphique



Lecture : l'axe horizontal est logarithmique pour montrer les tailles, très différentes, sur un même graphique. L'axe vertical reste linéaire. Les points correspondent aux mesures du tableau de la première page ; les lignes servent uniquement à guider la comparaison.

Interprétation et limites

Sur cette série, les durées augmentent globalement avec la taille de l'entrée. Pour un million de valeurs, l'insertion prend $975,218$ ms, la recherche $845,118$ ms et la suppression $909,071$ ms. À taille égale, la recherche est ici la plus rapide pour les grands tests. Une opération d'ABR dépend toutefois de la hauteur h de l'arbre : son coût est $O(h)$. Un arbre équilibré donne une hauteur proche de $\log_2(n)$, tandis qu'un arbre très désequilibré peut atteindre une hauteur de l'ordre de n.

Les valeurs sont mélangées, mais l'ABR n'est pas auto-équilibré. Les temps dépendent donc de l'ordre aléatoire, de l'environnement logiciel et de la charge de l'ordinateur. Une seule exécution par taille ne permet pas de calculer une moyenne ni de conclure à une loi de croissance exacte. Pour renforcer l'étude, répéter chaque taille plusieurs fois, puis comparer la médiane et la moyenne.

À noter : la carte graphique n'intervient pas dans cet algorithme séquentiel ; le benchmark sollicite principalement le processeur et la mémoire.

Reproduire le test

Depuis le dossier contenant le fichier C++ :

```
g++ -std=c++17 -Wall -Wextra -pedantic .\arbre_recherche_binaire.cpp -o .\
    arbre_recherche_binaire.exe
.\arbre_recherche_binaire.exe
```

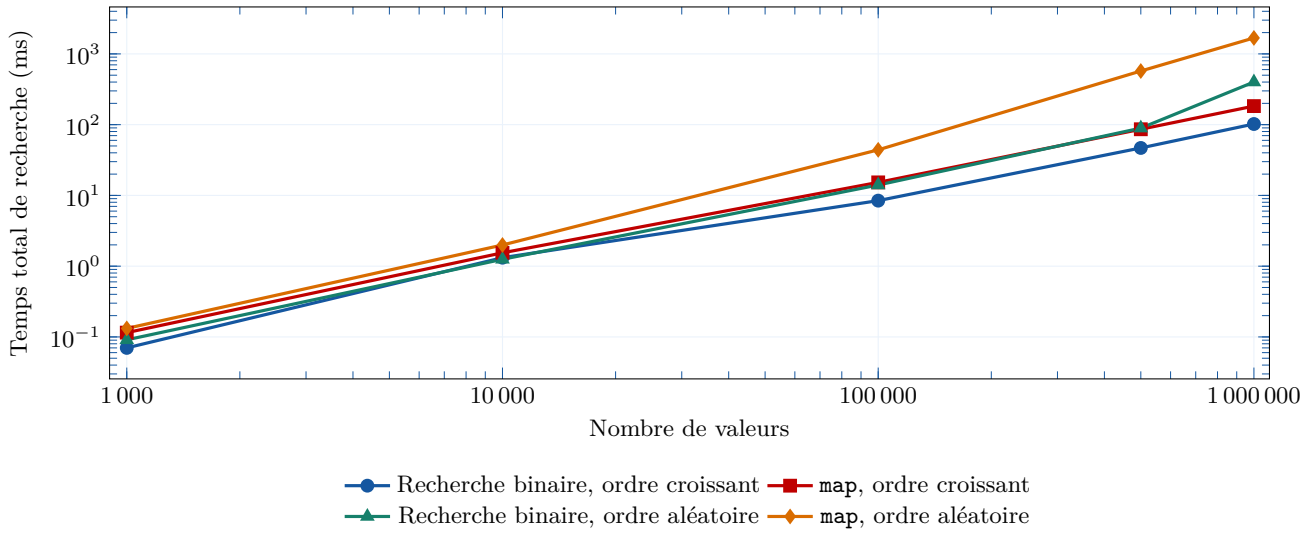
Dans le menu, choisir l'option `3`, saisir successivement les tailles du tableau, puis quitter avec `0`. Les temps peuvent varier d'un lancement à l'autre, puisque l'ordre des valeurs est mélangé aléatoirement.

Comparaison : recherche binaire et `std::map`

Une seconde expérience compare la recherche binaire dans un `std::vector` trié à la recherche par `std::map::find`. Le menu propose deux ordres d'entrée : valeurs croissantes ou valeurs mélangées. Les requêtes utilisent le même ordre que les insertions dans chaque scénario. Le mélange est reproductible grâce à une graine fixe. Dans le menu, choisir l'option `4`, puis `1` pour l'ordre croissant ou `2` pour l'ordre aléatoire et saisir la taille du test. Les valeurs générées sont les entiers de 0 à $n-1$. L'ordre d'insertion de `std::map` varie selon le scénario, mais le conteneur conserve dans les deux cas ses clés classées.

Pour le vecteur, le temps de préparation comprend la copie des valeurs et leur tri ; le temps de recherche comprend n appels à `std::binary_search`. Pour la map, le temps de construction comprend n insertions dans un `std::map<int, bool>`, et le temps de recherche comprend n appels à `find`. Les durées de construction et de recherche sont mesurées séparément. Chaque scénario a retrouvé toutes ses valeurs.

Temps de recherche



n	Ordre	Préparation vecteur (ms)	Construction map (ms)	Recherche binaire (ms)	Recherche map (ms)
1 000	Croissant	0,022	1,244	0,070	0,115
1 000	Aléatoire	0,058	0,236	0,091	0,132
10 000	Croissant	0,360	2,902	1,318	1,545
10 000	Aléatoire	0,723	3,595	1,251	1,992
100 000	Croissant	3,728	26,183	8,437	15,283
100 000	Aléatoire	8,796	40,967	14,033	44,062
500 000	Croissant	19,838	143,761	46,863	86,097
500 000	Aléatoire	52,344	456,767	89,380	572,895
1 000 000	Croissant	44,081	306,567	102,142	182,987
1 000 000	Aléatoire	115,485	1 816,966	400,506	1 681,062

Interprétation

Dans ces mesures, le vecteur trié est plus rapide en recherche que `std::map` dans les deux ordres, en particulier pour le scénario aléatoire à un million d'éléments. La recherche binaire divise l'intervalle de recherche par deux à chaque comparaison et a une complexité $O(\log n)$. La recherche dans `std::map` est également $O(\log n)$, mais ses noeuds sont répartis dynamiquement et la navigation par pointeurs a un coût pratique plus élevé ici.

La préparation du vecteur inclut le tri, mais ce coût n'est pas ajouté au temps de recherche du graphique. Le tableau permet donc de distinguer construction et requêtes. Pour une utilisation où les données sont construites une fois puis recherchées souvent, ces deux phases doivent être interprétées séparément. Les résultats restent ceux d'une exécution par scénario et peuvent varier avec le système et le compilateur.

Compilation utilisée : `g++ -std=c++17 -Wall -Wextra -pedantic` (sans option `-O` d'optimisation explicite).